

ASSIGNMENT – 3

1. What is exception? List different types of exception.

Answer:

In Python, an **exception** is an error that occurs during the execution of a program and disrupts the normal flow of the program. Exceptions occur due to invalid operations such as dividing by zero, accessing an invalid index, or opening a non-existing file.

Why exceptions occur

- Invalid user input
- Logical errors
- File handling errors
- Memory or type errors

Common types of exceptions in Python

1. **ZeroDivisionError** – division by zero
2. **ValueError** – wrong value type
3. **TypeError** – incompatible data types
4. **IndexError** – invalid index
5. **KeyError** – invalid dictionary key
6. **FileNotFoundError** – file not found

Conclusion:

Exceptions help identify runtime errors and allow programmers to handle them safely.

2. Explain how to use try, except, else and finally with proper example.

Answer:

In Python, exception handling is done using **try**, **except**, **else**, and **finally** blocks to handle runtime errors gracefully.

Explanation

- **try**: contains risky code
- **except**: handles the error
- **else**: executes if no exception occurs
- **finally**: always executes

Syntax:

```
try:  
    statements
```

```
except ExceptionType:
    statements
else:
    statements
finally:
    statements
```

Example:

```
try:
    a = int(input("Enter number: "))
    b = int(input("Enter number: "))
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("Division successful")
finally:
    print("Program ended")
```

Conclusion:

These blocks ensure smooth execution and proper error handling in Python programs.

3. What is raising an exception? Explain.

Answer:

In Python, **raising an exception** means forcing an error manually using the `raise` keyword when a specific condition occurs.

Why raise exceptions

- Custom error handling
- Input validation
- Enforcing program rules

Syntax:

```
raise ExceptionType("Error message")
```

Example:

```
age = int(input("Enter age: "))
if age < 18:
    raise ValueError("Age must be 18 or above")
else:
    print("Eligible")
```

Conclusion:

Raising exceptions gives developers control over error handling and improves program reliability.

4. Explain user-defined exception.**Answer:**

In Python, a **user-defined exception** is a custom exception created by the programmer to handle specific application-related errors.

Steps to create user-defined exception

1. Create a class
2. Inherit from `Exception` class
3. Use `raise` keyword

Syntax:

```
class MyException(Exception):  
    pass
```

Example:

```
class InvalidMarks(Exception):  
    pass  
  
marks = int(input("Enter marks: "))  
if marks > 100:  
    raise InvalidMarks("Marks cannot exceed 100")
```

Conclusion:

User-defined exceptions improve clarity and make programs more meaningful and secure.

5. Explain about init constructor.**Answer:**

In Python, `__init__` is a **constructor** method that automatically executes when an object of a class is created. It is used to initialize instance variables.

Features

- Automatically called
- Used for initialization
- Runs once per object

Syntax:

```
class ClassName:
    def __init__(self, parameters):
        statements
```

Example:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
s1 = Student("Aman", 19)
print(s1.name, s1.age)
```

Conclusion:

The `__init__` constructor initializes object data when the object is created.

6. Explain object creation and class creation.**Answer:**

In Python, a **class** is a blueprint, and an **object** is an instance of that class.

Class Creation

A class is created using the `class` keyword.

Syntax:

```
class ClassName:
    statements
```

Object Creation

An object is created by calling the class name.

Syntax:

```
object_name = ClassName()
```

Example:

```
class Car:
    def show(self):
        print("This is a car")
```

```
c1 = Car()
```

```
c1.show()
```

Conclusion:

Classes define structure, and objects represent real-world entities.

7. What are instance variables and class variables? Explain.**Answer:**

In Python, variables inside a class are of two types: **instance variables** and **class variables**.

Instance Variables

- Belong to object
- Defined using `self`
- Unique for each object

Class Variables

- Shared among all objects
- Defined outside methods

Example:

```
class Student:
    college = "ABC College"    # class variable
    def __init__(self, name):
        self.name = name      # instance variable
```

Conclusion:

Instance variables store object-specific data, while class variables store common data.

8. What is inheritance? Explain with its types.**Answer:**

In Python, **inheritance** allows a class to acquire properties and methods of another class.

Benefits

- Code reuse
- Reduces redundancy
- Improves readability

Syntax:

```
class ChildClass(ParentClass):
    statements
```

Types of Inheritance

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

Example:

```
class Parent:
    def show(self):
        print("Parent class")

class Child(Parent):
    pass

c = Child()
c.show()
```

Conclusion:

Inheritance promotes reusability and structured programming.

9. Explain use of super keyword.

Answer:

In Python, the **super()** keyword is used to access methods and variables of the parent class from the child class.

Uses of super()

- Call parent constructor
- Access parent methods
- Avoid method overriding issues

Syntax:

```
super().method_name()
```

Example:

```
class Parent:
    def __init__(self):
        print("Parent constructor")

class Child(Parent):
    def __init__(self):
```

```
super().__init__()  
print("Child constructor")
```

```
c = Child()
```

Conclusion:

The `super` keyword helps achieve proper inheritance and method access.